

Software Design and Architecture

Lecture-1

By: Usama Musharaf

A solid blue wave-like shape that spans the width of the slide, positioned at the bottom.

Course Learning Outcomes

- 1) Understand the fundamentals of software requirements analysis, design and architecture, and process models.
- 2) Understand different software architectural styles used for developing software systems.
- 3) Recognize the importance of quality attributes and apply architectural tactics to achieve system quality.
- 4) Apply software design concepts to design and implement software systems using an object-oriented programming language.
- 5) Understand various design principles and patterns to improve the quality of software design.

What is Software?

According to the IEEE

- Software is:
 - *“Computer programs, associated documentation and data pertaining to the operation of a computer system”.*

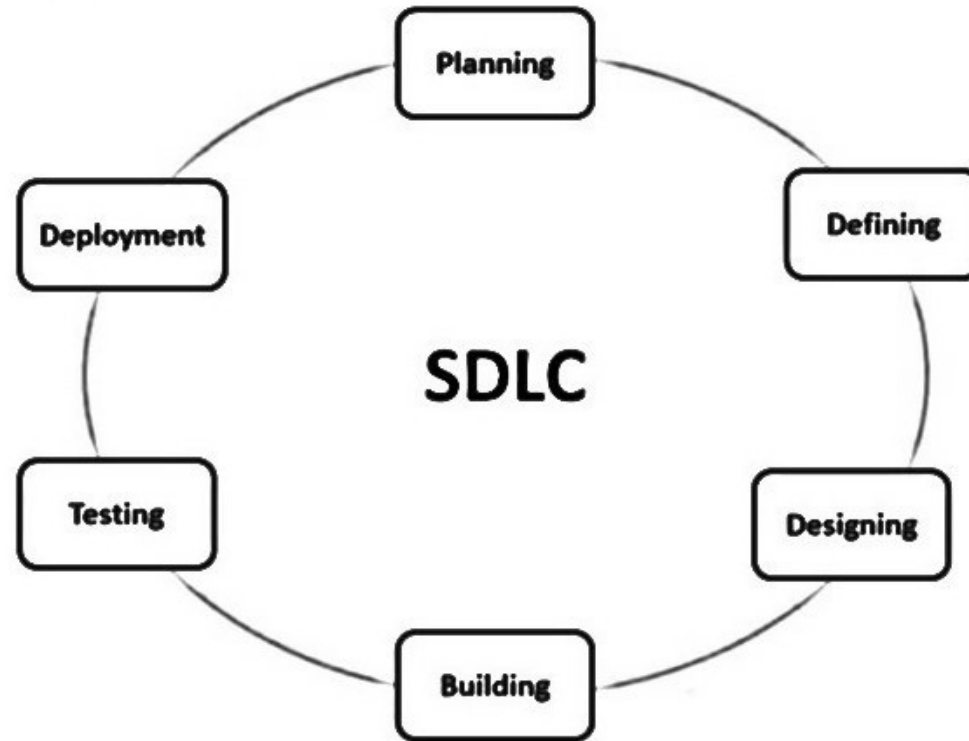
Software Engineering

- Systematic approach for developing software.
- Methods and techniques to develop and maintain quality software to solve problems.
- Study of the principles and methodologies for developing and maintaining software systems.

Software Engineering



Software Development Life Cycle



Software Engineering Phases

- Definition: What?
- Development: How?

Problems in Software Development

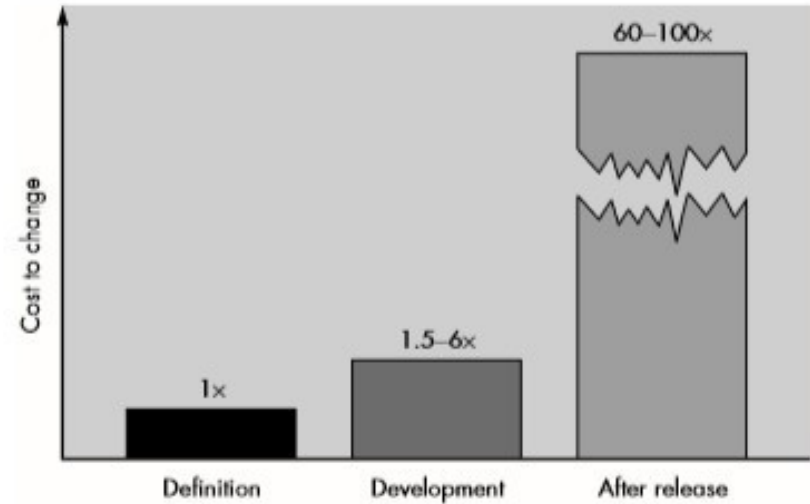
Common issues

- The final software does not fulfill the needs of the customer.
- Hard to extend and improve: if you want to add a functionality later its mission impossible.
- Bad documentation.
- Bad quality: frequent errors, hard to use, ...
- More time and costs than expected.

Problems in Software Development

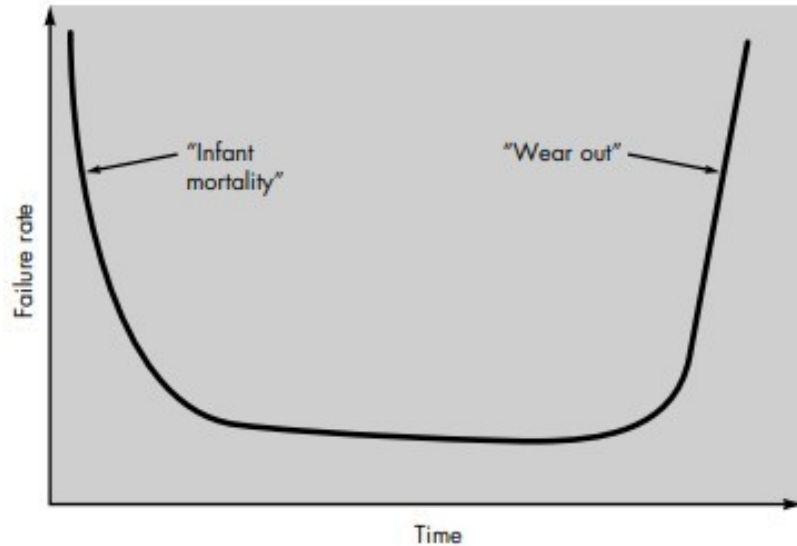
Changing Requirements

- 5 x cost during development
- up to 100 x cost during maintenance

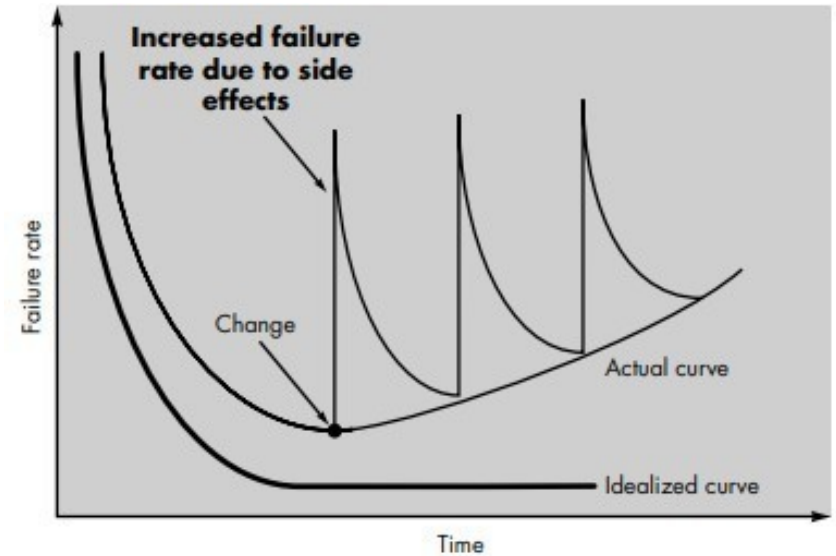


Failure Curve

Failure curve for hardware



Idealized and actual failure curves for software



WHAT ARE THE ATTRIBUTES OF GOOD SOFTWARE?

SOFTWARE DESIGN

Software Design

- Design is the first step in the development phase for any engineered product or system.
- Design is about HOW the system will perform its functions.

Software Design

- A software design is a meaningful engineering representation of some software product that is to be built.
- The process of applying various techniques and principles for the purpose of defining a device, a process or a system in sufficient detail to permit its physical realization.

Software Design

- *Requirements specification* was about the WHAT the system will do
- *Design* is about the HOW the system will perform its functions
 - provides the overall decomposition of the system.
 - allows to split the work among a team of developers.
 - also lays down the groundwork for achieving non-functional requirements (performance, maintainability, reusability, etc.)

Motivation



Software Design Process Activities

- Architectural design
- Interface design
- Component design
- Data structure design
- Algorithm design

Levels of Software Design

Architectural design (high-level design)

- architecture - the overall structure, main modules and their connections.
- addresses the main non-functional requirements (e.g., reliability, performance) hard to change.

Detailed design (low-level design)

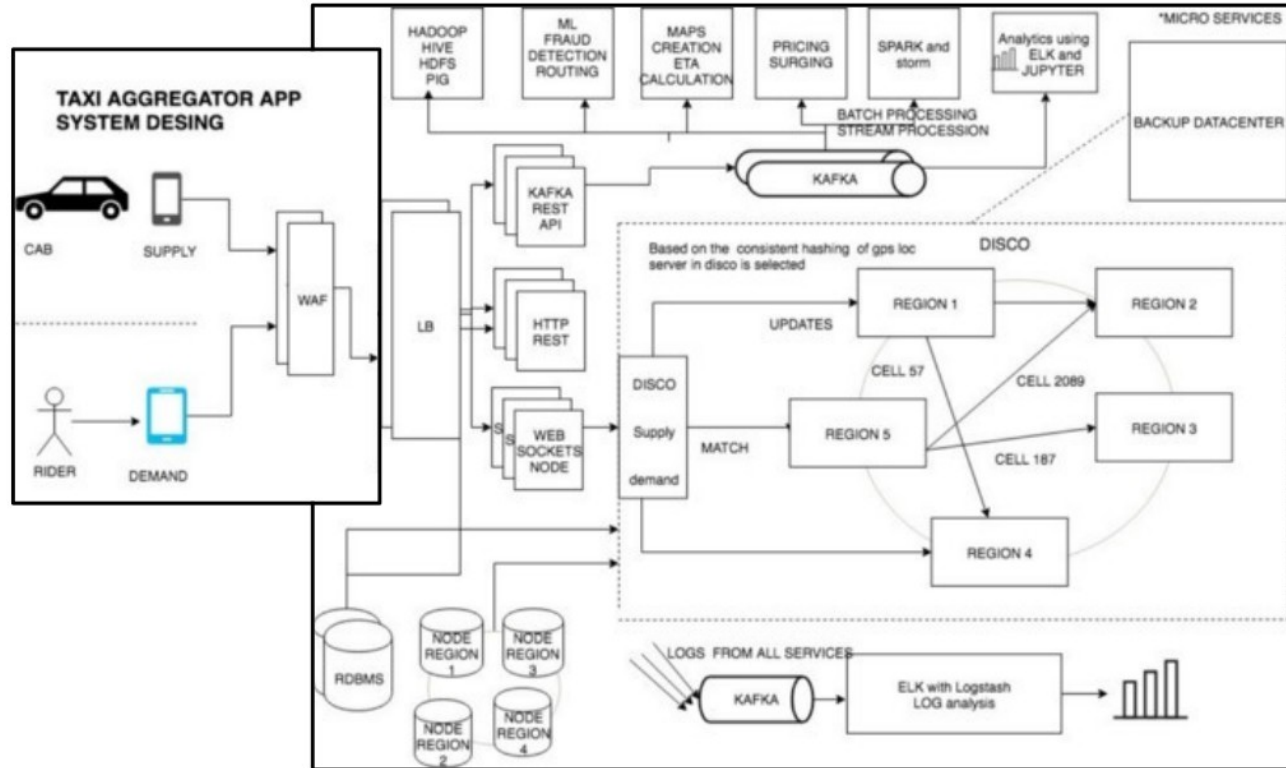
- the inner structure of the main modules
- detailed enough to be implemented in the programming language

Software Architecture (High Level Design)

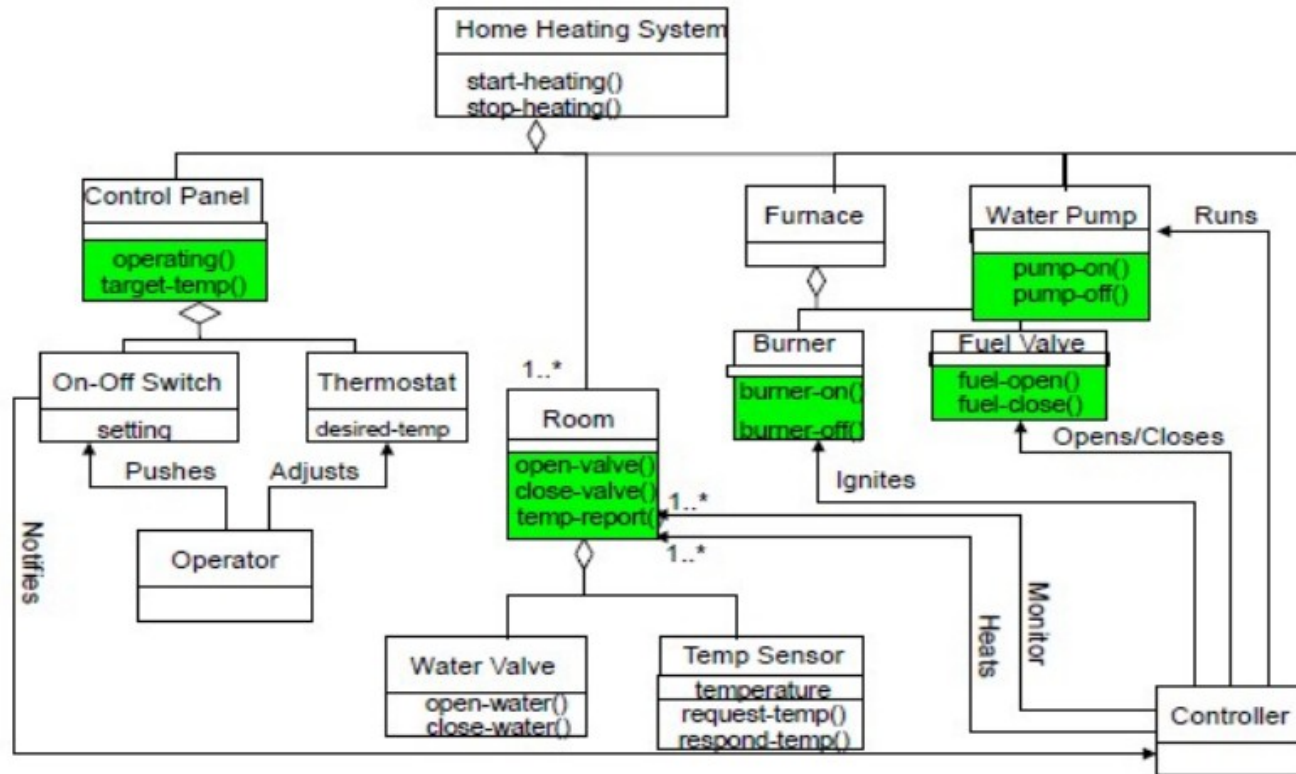
Software architecture is the high-level structure of a software system that defines how the system is organized, how its major components interact, and the principles and decisions that guide its design and evolution.

- What are the main parts of the system?
- How do these parts communicate?
- Why was this structure chosen?

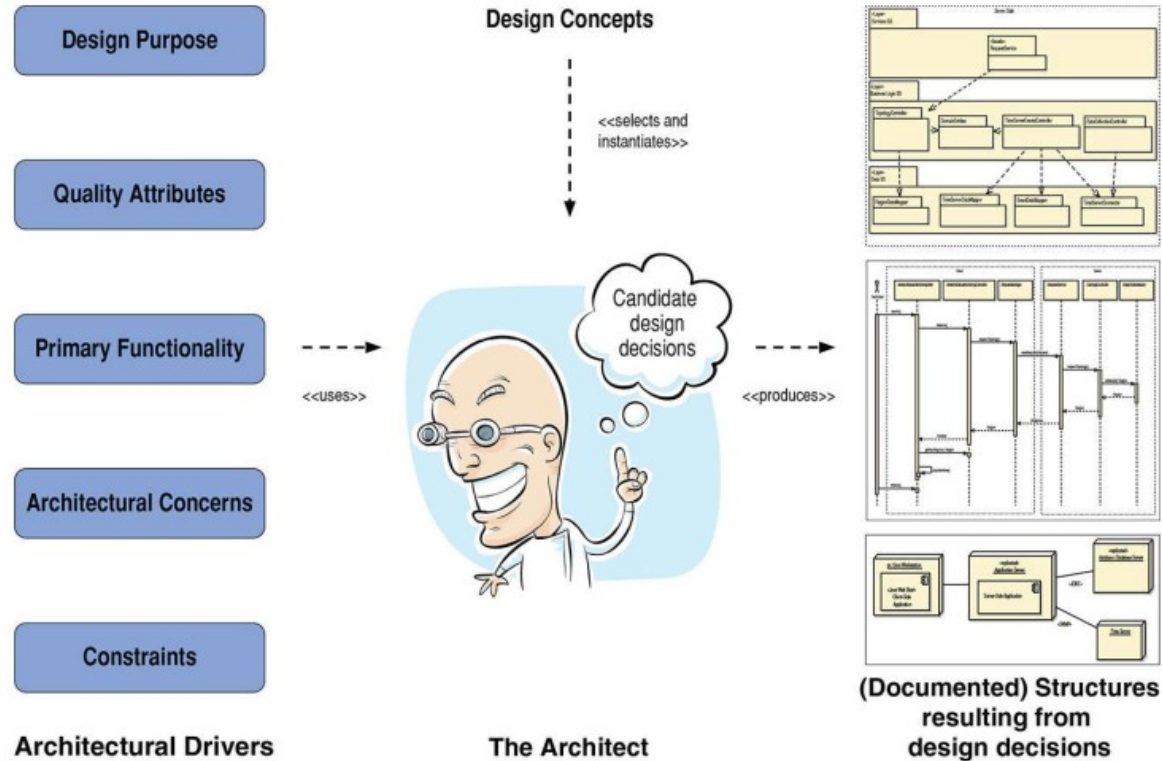
Software Architecture Example



Detailed Design (Low Level)



Overview of Architecture Design Activity



Architectural Drivers

1-Design Purpose:

The overall goal of the system

Why the system is being built

Example: real-time monitoring, data processing,

2-Quality Attributes

Non-functional requirements

Examples: performance, scalability, availability, security, maintainability

These strongly influence architectural choices.

3-Primary Functionality

Core system features and services

What the system must deliver to users

Example: request handling, data storage, analytics

4- Architectural Concerns

Key risks, trade-offs, and technical challenges

Examples: concurrency, fault tolerance, data consistency

5- Constraints

Limitations that restrict design freedom

Examples: budget, legacy systems, technology stack,

SOFTWARE PROCESS MODELS

Software Process Models

Waterfall

V Model

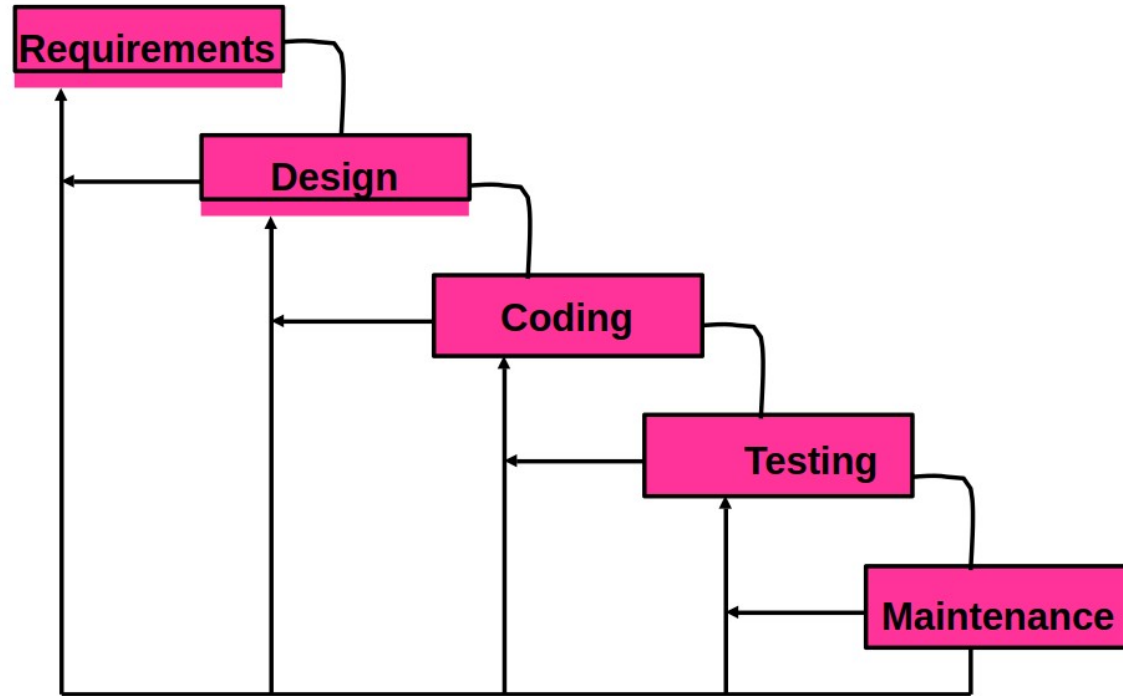
Incremental

Spiral

RUP

Agile Development

Waterfall (Linear Sequential)



Waterfall (Linear Sequential)

Advantages

- Simple and easy to understand and use
- Works well for smaller projects where requirements are very well understood.
- Clearly defined stages.
- Well understood milestones.
- Easy to arrange tasks.
- Process and results are well documented.

Waterfall (Linear Sequential)

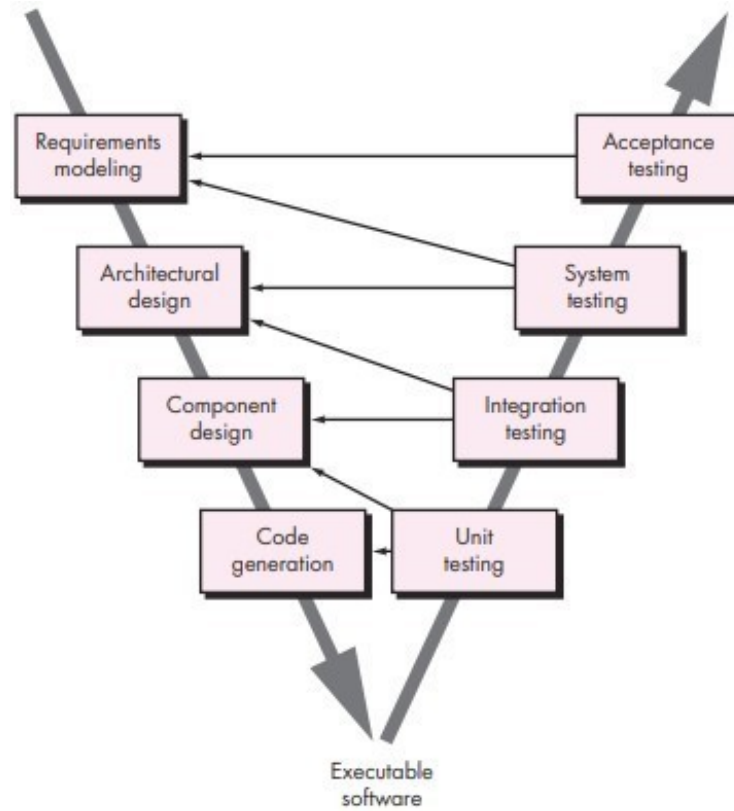
Disadvantages:

- The drawback of the waterfall model is the difficulty of accommodating change after the process is underway.

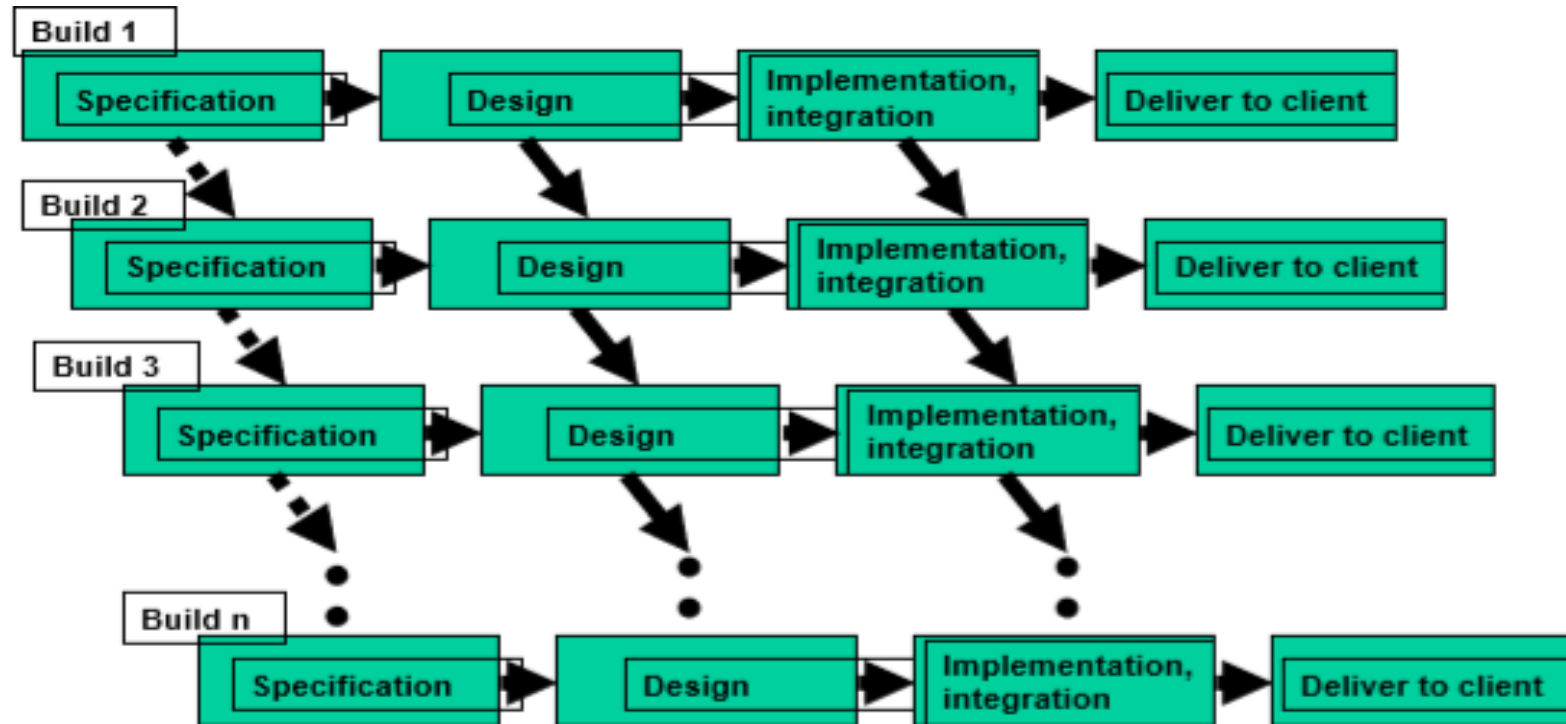
This makes it difficult to respond to changing customer requirements

Therefore, this model is only appropriate when the requirements are well-understood.
- Project failure risk is high.

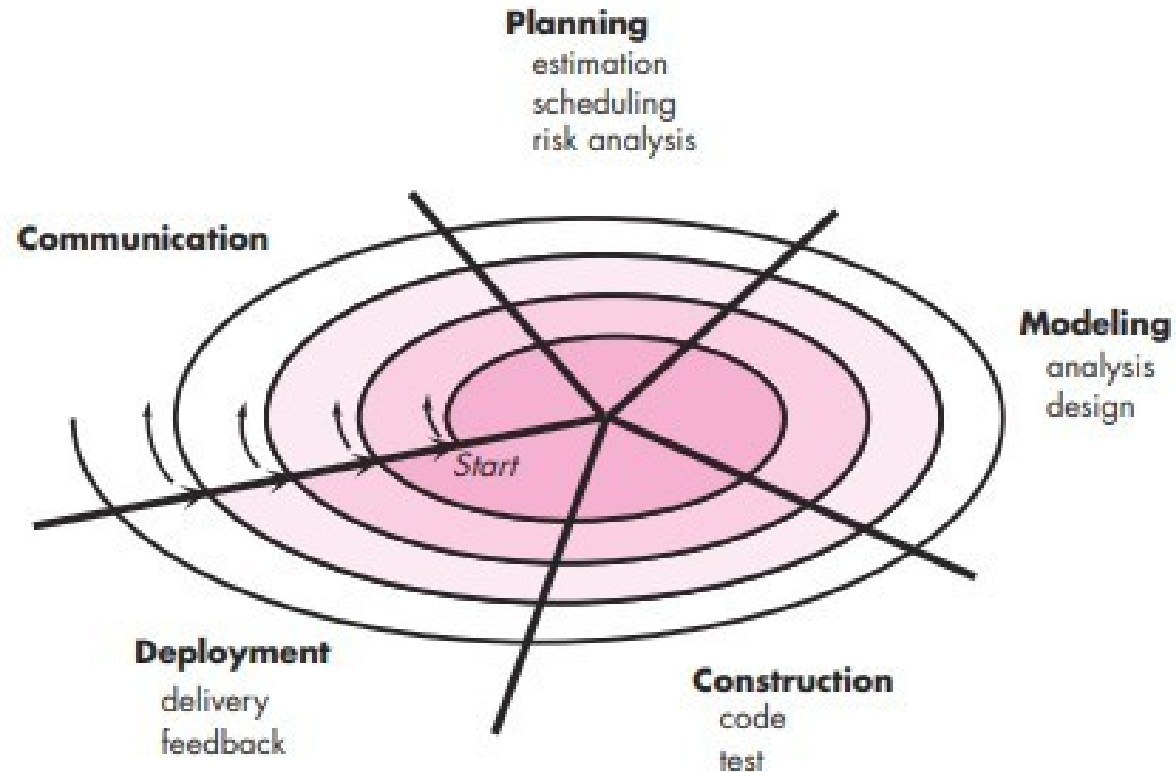
V Model



Incremental Model



Spiral Model



RUP Model

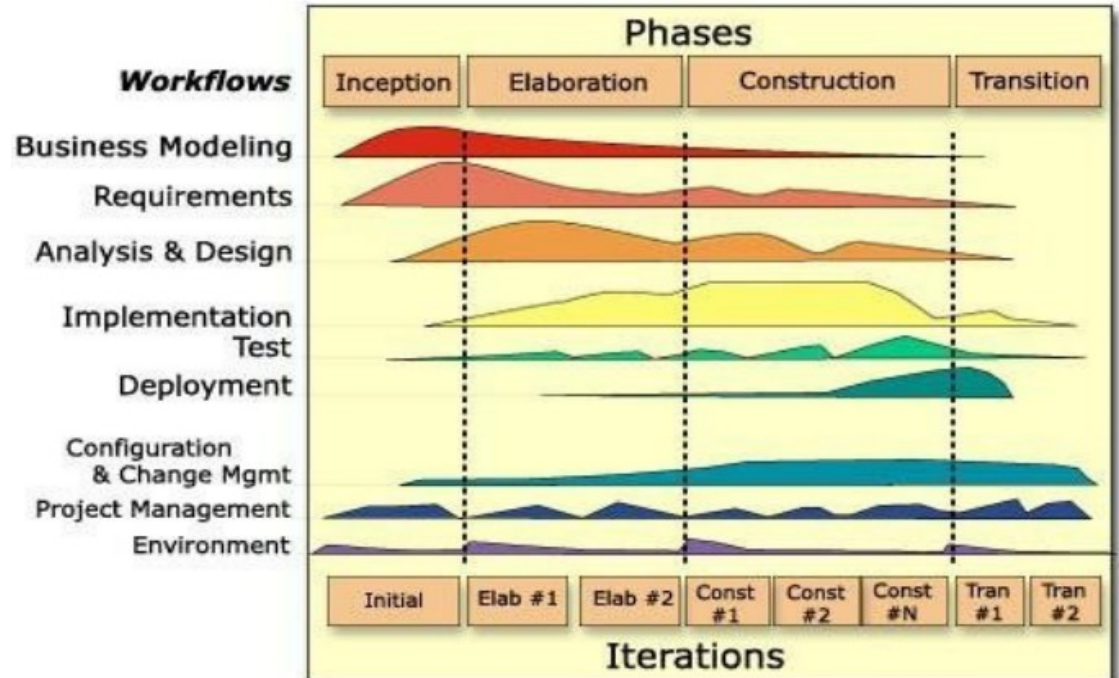
Rational Unified Process

Iterative and Incremental: Development proceeds in iterations, delivering working software incrementally.

Use-Case Driven: Functional requirements are captured and validated using use cases.

Architecture-Centric: Early focus on defining and stabilizing the software architecture.

Risk-Driven: High-risk elements are addressed early in the project.



RUP Model

1. Inception Phase

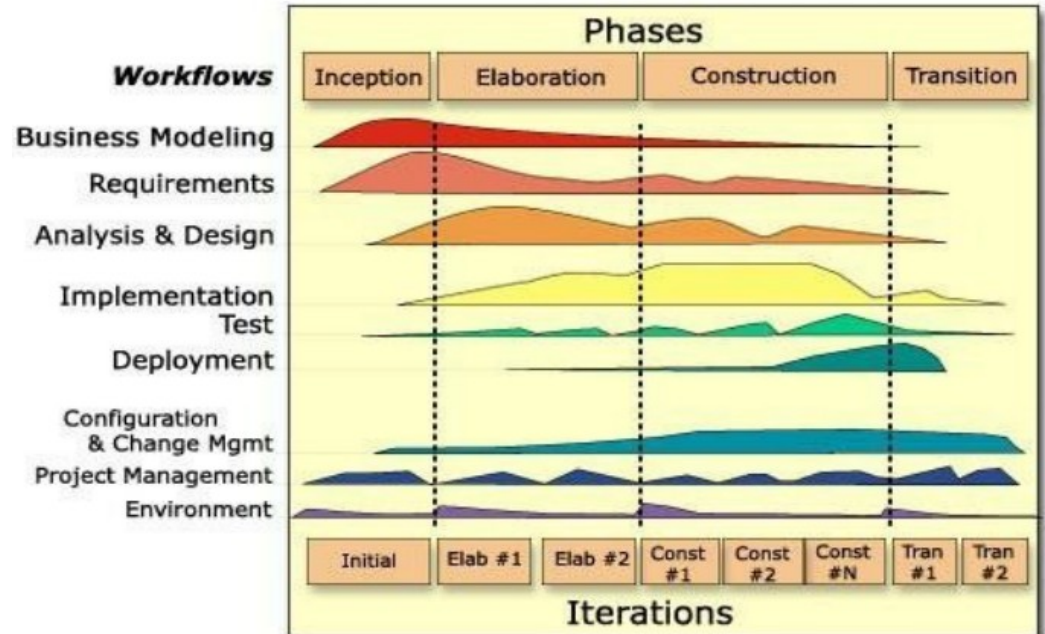
Purpose: Define the project scope and feasibility.

Activities:

- Identify stakeholders
- Capture high-level requirements (use cases)
- Identify major risks
- Prepare initial project plan

Deliverables:

- Initial use-case model
- Risk list



RUP Model

2. Elaboration Phase

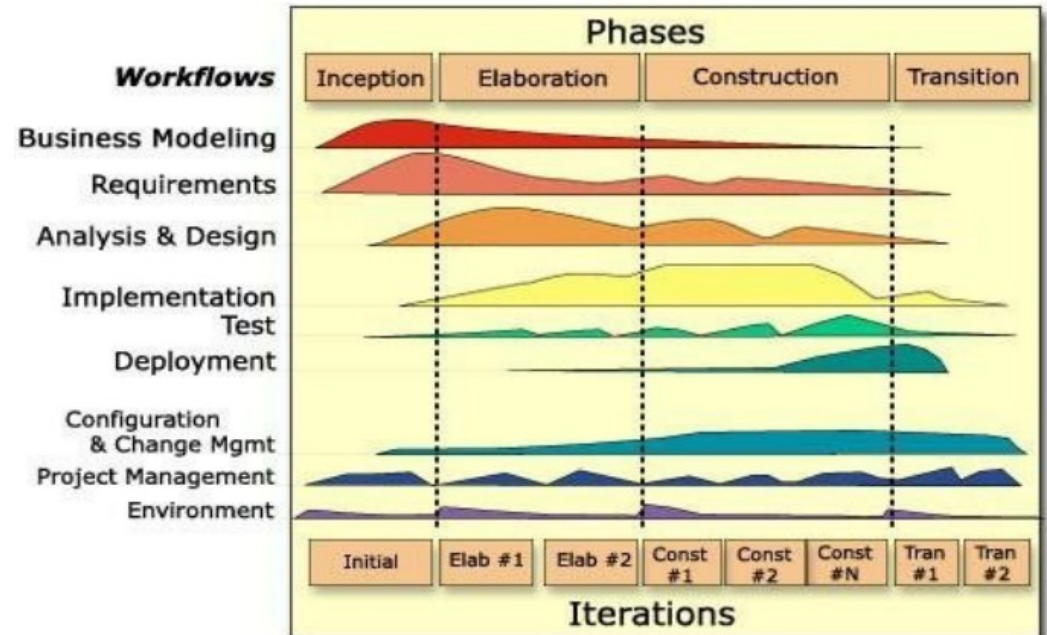
Purpose: Establish a solid architectural foundation.

Activities:

- Refine requirements in detail
- Identify and validate architectural decisions
- Address highest technical risks
- Create executable architectural prototype

Deliverables:

- Software Architecture Document (SAD)
- Refined use-case model
- Updated project plan
- Risk mitigation strategies



RUP Model

3. Construction Phase

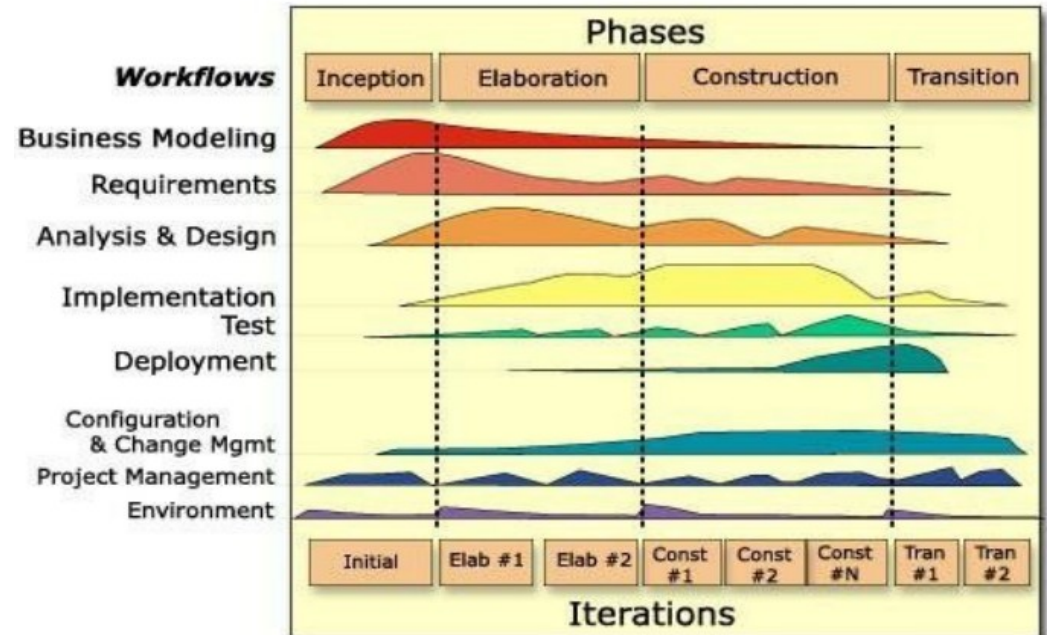
Purpose: Build the complete system.

Activities:

- Implement system functionality
- Perform unit and integration testing
- Continuous integration
- Iterative development of features

Deliverables:

- Executable software increments
- Test cases and test results
- User documentation drafts



RUP Model

4. Transition Phase

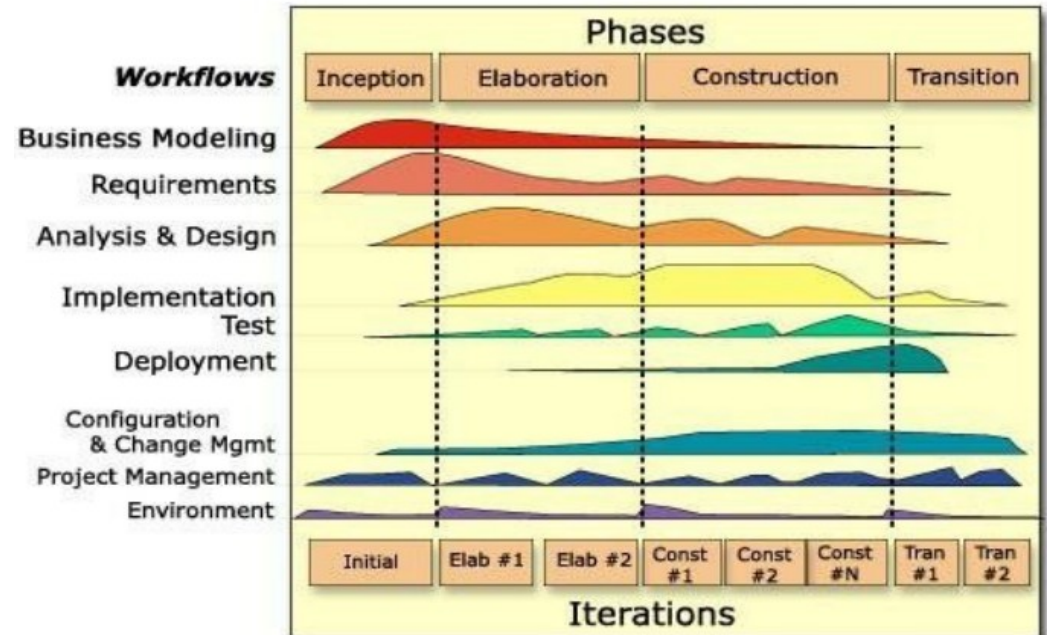
Purpose: Deliver the system to users.

Activities:

- System and acceptance testing
- User training
- Deployment and release
- Bug fixing and fine-tuning

Deliverables:

- Final software product
- User manuals



AGILE DEVELOPMENT

Agile Development

Agile is a philosophy or a way of thinking guided by some values and principles.

Agility:

Effective (rapid and adaptive) response to change

Effective communication among all stakeholders

Yielding ...

Rapid, incremental delivery of software.

Agile Values

1. Individuals and Interactions over Processes and Tools
2. Working Software over Comprehensive Documentation
3. Customer Collaboration over Contract Negotiation
4. Responding to change over Following a Plan

Agile Process Models

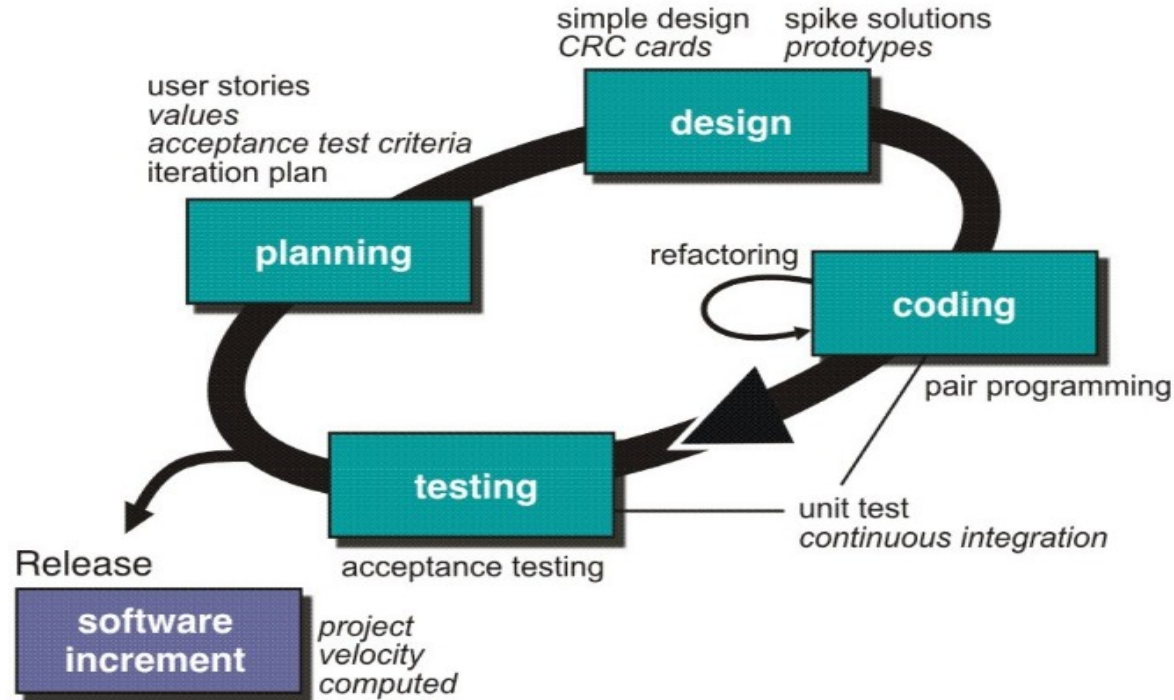
1. Extreme Programming
2. Scrum

Extreme Programming

Extreme Programming (XP) takes an 'extreme' approach to iterative development.

- New versions may be built several times per day;
- Increments are delivered to customers every 2 weeks;
- All tests must be run for every build and the build is only accepted if tests run successfully.

Extreme Programming



Extreme Programming

XP Planning

- Begins with the creation of user stories
- Agile team assesses each story and assigns a cost
- Stories are grouped to for a deliverable increment
- A commitment is made on delivery date

Extreme Programming

User Story Card:

Downloading and printing an article

First, you select the article that you want from a displayed list. You then have to tell the system how you will pay for it - this can either be through a subscription, through a company account or by credit card.

After this, you get a copyright form from the system to fill in and, when you have submitted this, the article you want is downloaded onto your computer.

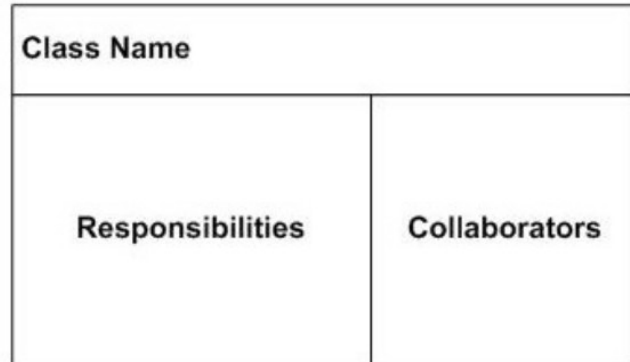
You then choose a printer and a copy of the article is printed. You tell the system if printing has been successful.

If the article is a print-only article, you can't keep the PDF version so it is automatically deleted from your computer .

Extreme Programming

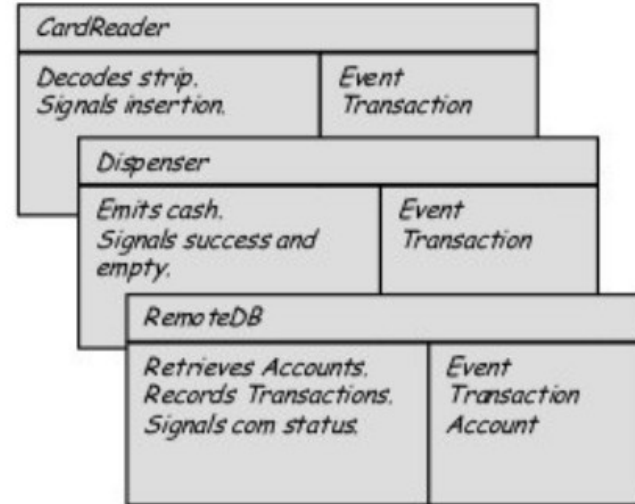
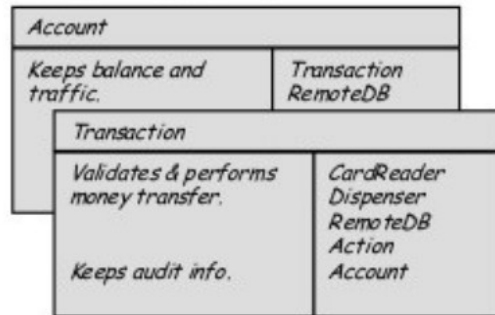
CRC Cards for Design:

Class-responsibility-collaboration (CRC) cards are a tool used in the design of object-oriented software.



Extreme Programming

CRC Cards for ATM:

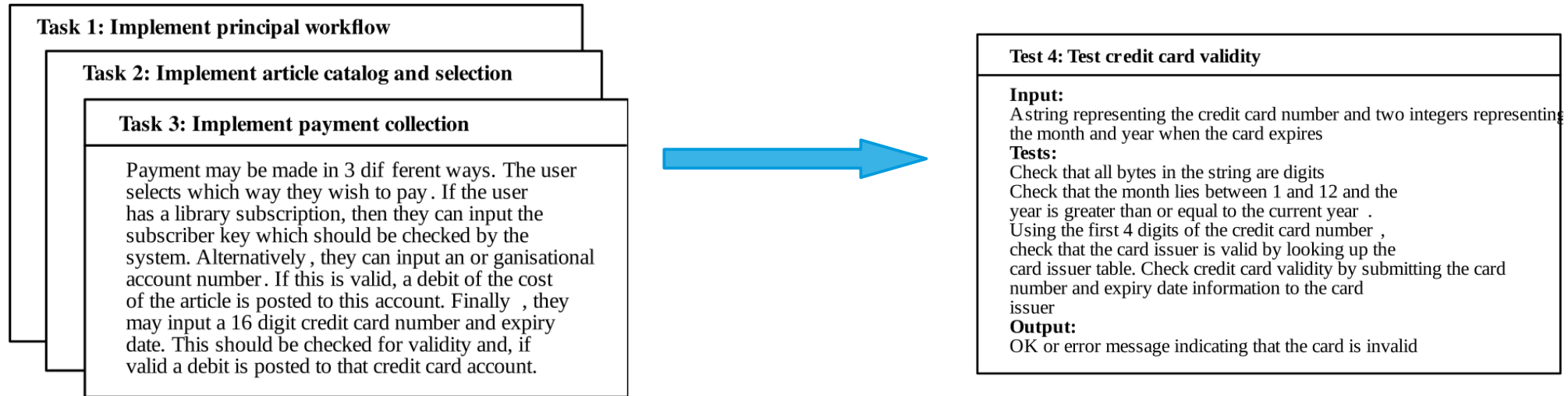


Extreme Programming

Testing:

Test-first development.

Incremental test development from scenarios.



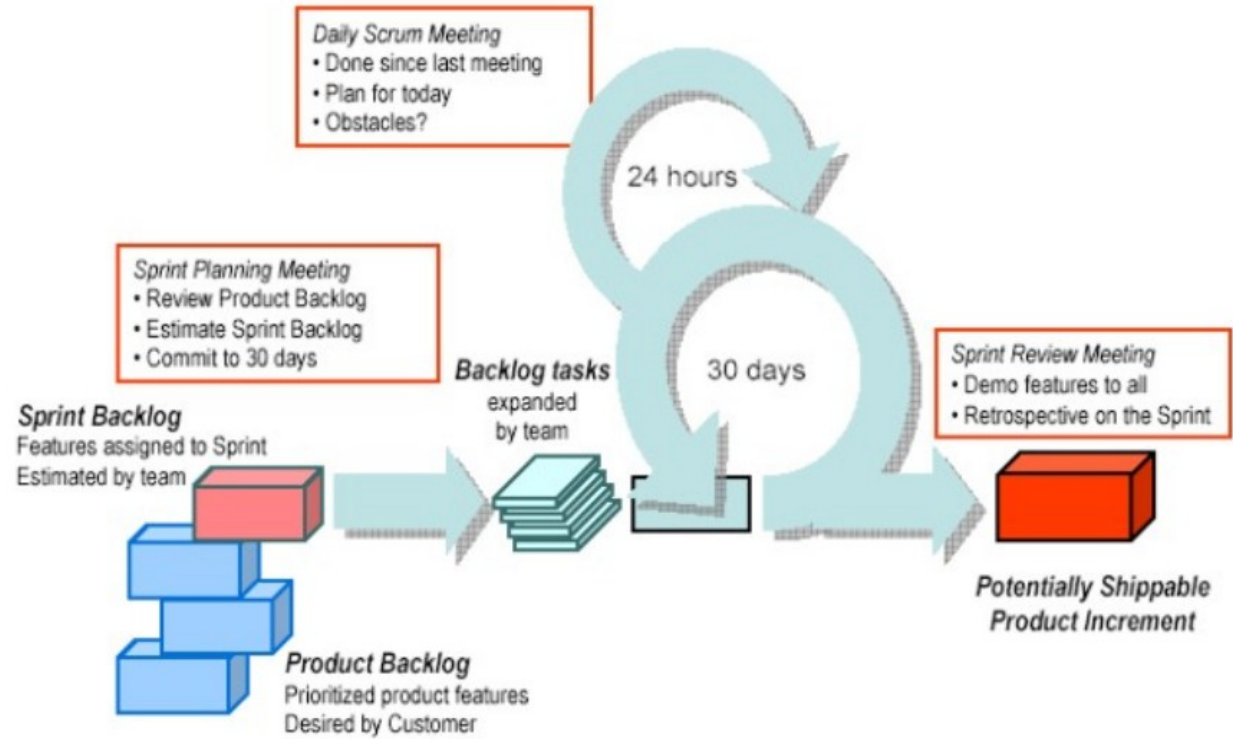
Scrum

Scrum is an Agile framework for completing complex projects.

Scrum originally was formalized for software development projects, but it works well for any complex, innovative scope of work.

Scrum is a team-based approach, to iteratively, incrementally develop systems and products.

When requirements are rapidly changing



Difference between XP and Scrum

Focus		
Aspect	XP	Scrum
Main Focus	Engineering Practices	Project Management and process
Emphasis	How to build software correctly	How to manage work and team

Difference between XP and Scrum

Role	
XP	Scrum
Programmer, Customer, Coach, Tracker, Tester	Product Owner, Scrum Master, Development Team

Difference between XP and Scrum

Iterations & Planning		
Aspect	XP	Scrum
Iteration length	1-2 weeks	2-4 weeks (sprint)
Scope change	Allowed anytime	Locked during sprint

Difference between XP and Scrum

XP Strongly Emphasizes:

- Test-Driven Development (TDD)
- Pair Programming
- Continuous Integration
- Refactoring
- Simple Design

Scrum

- Does not mandate engineering practices
- Teams choose how to code

When to use which?

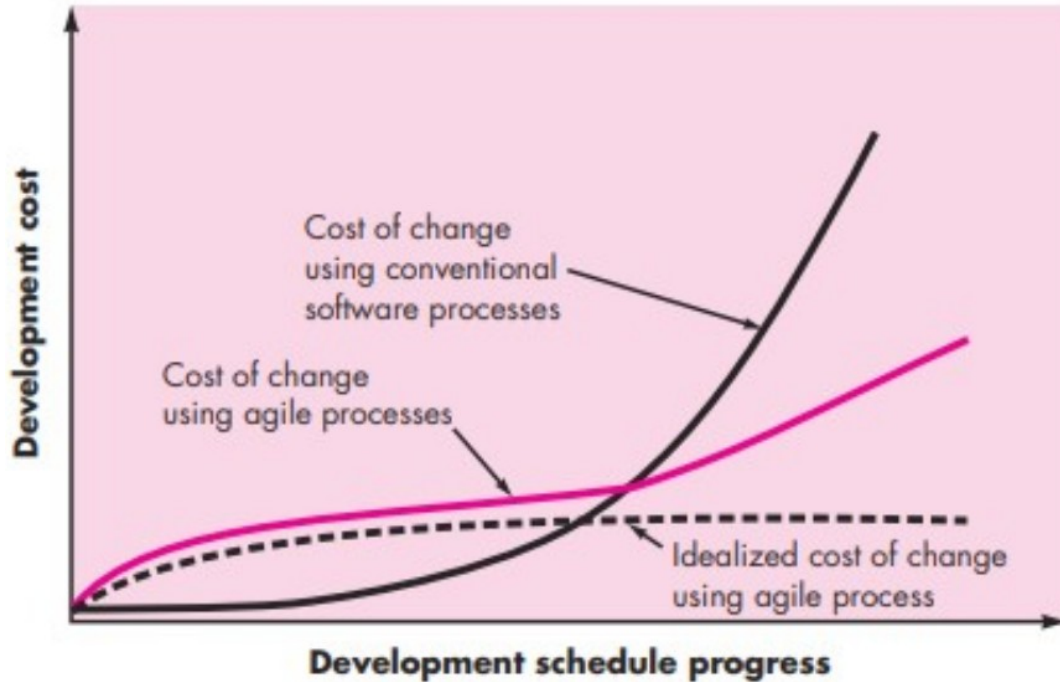
Use XP when:

- Requirements change frequently
- Code quality is critical
- You want strong technical discipline
- Small, highly skilled team

Use Scrum when:

- Team coordination is a challenge
- Project needs visibility & control
- Organization is new to Agile
- Larger teams or multiple teams

Cost of change in Agile



Case Study: Online Food Ordering

A startup wants to build an Online Food Ordering System where customers can:

- View restaurants and menus
- Place orders
- Track order status

Requirements are unclear and change frequently due to market feedback.

User Stories: Online Food Ordering

Story Card 1

Title: View Menu

As a customer

I want to view restaurant menus

So that I can choose food items

Acceptance Criteria

Menu shows item name, price, availability

Out-of-stock items are disabled

Story Card 3

Title: Place Order

As a customer

I want to place an order

So that food can be delivered

Acceptance Criteria

Order saved successfully

Order ID generated

Story Card 2

Title: Add Item to Cart

As a customer

I want to add food items to my cart

So that I can place an order

Acceptance Criteria

Cart updates quantity correctly

Total price updates automatically

User Stories: Online Food Ordering

Story Card 4

Title: Make Payment

As a customer

I want to pay online

So that my order is confirmed

Acceptance Criteria

Payment success confirms order

Payment failure cancels order

Story Card 6

Title: Cancel Order

As a customer

I want to cancel my order

So that I can stop unwanted delivery

Acceptance Criteria

Cancellation allowed before preparation

Refund initiated automatically

Story Card 5

Title: Track Order

As a customer

I want to track my order

So that I know delivery status

Acceptance Criteria

Status updates:

Placed → Preparing → Out for delivery → Delivered

CRC Cards: Online Food Ordering

Customer	
Place order Make payment Track order	Order Payment OrderTracker

Cart	
Add/remove items Calculate total price	Menu Order

Payment	
Process payment Validate transaction	Order Customer

Menu	
Display food items Provide prices and availability	Cart Order

Order	
Create order Store order details Manage order status	Cart Payment OrderTracker

OrderTracker	
Update order status Display delivery progress	Order Customer

Test Cards: Online Food Ordering

Test Card-1: Cart Total Calculation

Given: Cart has 2 burgers (500 each)
When: Total is calculated
Then: Total should be 1000

Test Card-2: Add Item to Cart

Given: Empty cart
When: Item added
Then: Cart size increases by 1

Test Card-3: Successful Order Placement

Given: Cart has items and payment succeeds
When: Order is placed
Then: Order ID is generated and status = "Placed"

Test Card-4: Payment Failure

Given: Payment fails
When: Order attempted
Then: Order is not confirmed

Test Card-5: Order Tracking

Given: Order is out for delivery
When: Customer checks status
Then: Status shows "Out for delivery"

Test Card-6: Order Cancellation

Given: Order status is "Placed"
When: Customer cancels
Then: Order is cancelled and refund initiated

DEVOPS

DevOps

DevOps aims to unify software development (Dev) and IT operations (Ops) to improve collaboration, reduce time-to-market, and increase software quality through automation and continuous delivery.



DevOps culture supports Agile processes



Why DevOps?

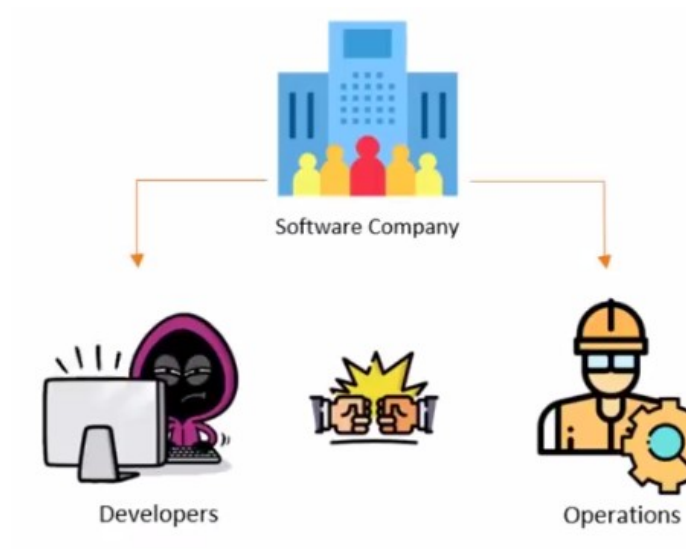
Collaboration

Reduce Conflict (Dev team + Ops team)

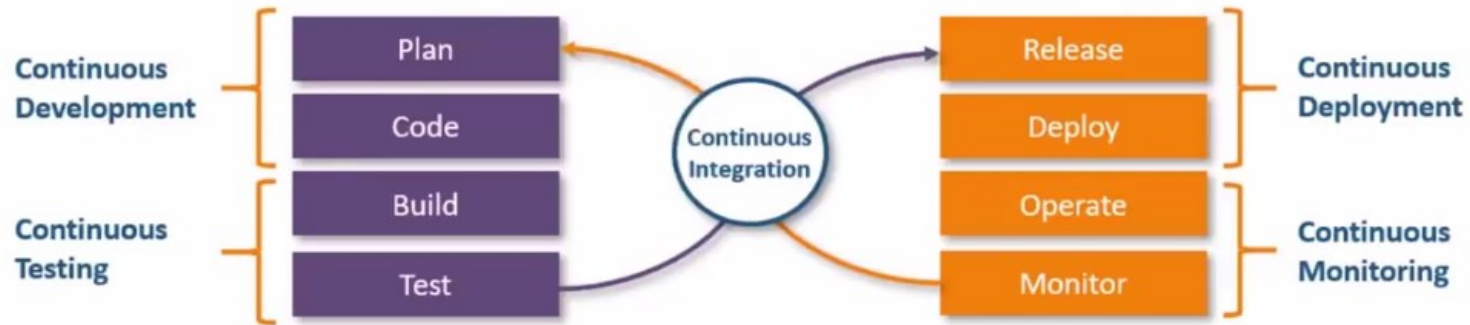
Faster Delivery (Pipelines)

Improved Quality

Reliability & Scalability



How DevOps Work?



Have A Good Day!